Formalizing the homotopy extension property in LEAN

Mara Masike Silge

Born on 24. April 2003 in Werne

August 12, 2026

Bachelor's Thesis Mathematics

Advisor: Prof. Dr. Floris van Doorn

Second Advisor: Prof. Dr. Markus Hausmann

Mathematisches Institut

Mathematisch-Naturwissenschaftliche Fakultät der

Rheinischen Friedrich-Wilhelms-Universität Bonn

Zusammenfassung

Motivation: Whitehead theorem oder so?

Contents

1 Vorbemerkung

In this work, if not specifized in any other way, every space has a topology and every map is a continuous map.

1.1 Motivation

The goal of this thesis is to formalize the *homotopy extension property (HEP)* in Lean. I started with the proof of the *retraction criterion* and worked towards the result that every relative CW complex satisfies the HEP. Our motivation for studying this property is that it is needed for two major theorems in algebraic topology. The first is the *cellular approximation theorem.*

Theorem 1.1 (Cellular Approximation Theorem, [**MR1867354**])**.** *Every map $f\colon X \to Y$ of CW complexes is homotopic to a cellular map. If f is already cellular on a subcomplex $A \subset X$, the homotopy may be taken to be stationary on A.*

Here a map is called *cellular* if it sends the n-skeleton of X to the n-skeleton of Y for all $n \in \mathbb{N}$.
Equipping S^k with the CW structure consisting of one 0-cell and one k-cell, it has no cells in dimensions $0 < n < k$. Hence any map $S^n \to S^k$ with $n < k$ is homotopic to one landing in the n-skeleton, which is a single point, and is therefore nullhomotopic. This yields the corollary $\pi_n(S^k) = 0$ for all $n < k$.

Secondly, the result that relative CW complexes satisfy the HEP is one of the main steps towards the proof of *Whitehead's theorem.*

Theorem 1.2 (Whitehead's Theorem, [**MR1867354**])**.** *If a map $f\colon X \to Y$ between connected CW complexes induces isomorphisms $f_*\colon \pi_n(X) \to \pi_n(Y)$ for all n, then f is a homotopy equivalence. In case f is the inclusion of a subcomplex $X \hookrightarrow Y$, the conclusion is stronger: X is a deformation retract of Y.*

Whitehead's theorem states that, in order to prove that two connected CW complexes are homotopy equivalent, it suffices to exhibit a map between them that induces isomorphisms on all homotopy groups, i.e. a weak homotopy equivalence. Showing that two spaces are homotopy equivalent is in general a difficult task, and Whitehead's theorem provides a powerful tool for such problems.

2 Homotopy Extension Property

There is not really one singel source, which my formalisation is based on. But for the overall structure I used my lecture notes from the topology I couse, thaught by Markus Hausmann in the winter semester 2025/26 at the university of Bonn. They are not publically available, but especially the relevant chapter about the homotopy extension property comes close to the lecture notes "Algebraische Topologie" of F. Waldhausen from the university in Bielefeld. This source is in German.

2.1 The Homotopy Extension Property

There are several equivalent ways to define the homotopy extension property, which will be discussed in this first chapter. This fist definition is closest to what I formalized in Lean.

Definition 2.1. A pair of topological spaces (X, A) with $A \subseteq X$ has the *homotopy extension property (HEP)* if for every topological space Y, every map $f\colon X \to Y$, and every homotopy $H\colon A \times [0,1] \to Y$ satisfying $H(a,0) = f(a)$ for all $a \in A$, there exists a homotopy $H'\colon X \times [0,1] \to Y$ such that

$$\begin{aligned} H'(x,0) &= f(x) && \text{for all } x \in X, \\ H'(a,t) &= H(a,t) && \text{for all } (a,t) \in A \times [0,1]. \end{aligned}$$

Remark 2.2. Note, that for any space X, the pair (X, X) and the pair $(X, \emptyset)$ have the *HEP*.

Remark 2.3. Another equivalent way of phrasing this definition is to say that (X, A) has the *HEP* if and only if every map defined on $X \cup_A (A \times [0,1])$ extends to a map on $X \times [0,1]$. Note that this is, in general, not the same as requiring that every map defined on $X \times \{0\} \cup (A \times [0,1])$ extends to a map on $X \times [0,1]$. This is because the natural map

$$X \cup_A (A \times [0,1]) \to X \times \{0\} \cup (A \times [0,1])$$

is not necessarily a homeomorphism. In general, it is only a continuous bijection. However, if A is closed, then this map is a homeomorphism and therefore yields another equivalent definition.

The equivalece from theorem 2.3 is proofen in the following lemma.

Lemma 2.4. *Let A be a subset of a topological space X.*

(i) *(X, A) has the* HEP

(ii) *for every topological space Y and every continuous map $g\colon X \times \{0\} \cup A \times [0,1] \to Y$ there exists an extension of g to a map $G : X \times [0,1] \to Y$*

Then $(i) \implies (ii)$ *and under the additional condition, that* A *is closed, also the inverse direction* $(ii) \implies (i)$ *holds.*

Proof. **(i)** $\Rightarrow$ **(ii):** Let $g : X \times \{0\} \cup A \times [0,1] \to Y$. We define maps $f\colon X \to Y, x \mapsto g(x,0)$ and $H\colon A \times [0,1] \to Y, (x,t) \mapsto g(x,t)$. It is easy to check, that f and H agree on $A \times [0,1]$, so the *HEP* for the pair (X,A) yields an extension of those two maps. We can chose G to be exactly this extended homotopy.
(A **closed** $\wedge$ **(ii))** $\Rightarrow$ **(i):** Let $f\colon X \to Y$ and $H\colon A \times [0,1] \to Y$. We define the map

$$g\colon X \times \{0\} \cup A \times [0,1] \to Y, (x,t) \mapsto \begin{cases} f(x), & \text{if } t = 0, \\ H(x,t), & \text{else} \end{cases}$$

Clearly g is well-defined, since f and H agree on A. To proof continuity of g, we need that $A \subseteq X$ is closed:
$A \subseteq X$ closed $\implies$ $A \times [0,1] \subset X \times [0,1]$ closed. Furthermore, $X \times \{0\} \subseteq X \times [0,1]$ is closed. So g is continuous, because it is continuous on two closed subsets and therefor also on their union.
By assumption there exists an extension $G : X \times [0,1]$ of g, which we can choose to be the extended homotopy for the definition of the *HEP*. □

Lemma 2.5. *Let* A *be a subset of a topological space* X*. Then the following are equivalent:*

(i) for every $g\colon A \to Y$ *there exists an extension* $G\colon X \to Y$

(ii) A *is a retract of* X*, i.e.* $\exists r : X \to A$*, such that* r *is the identity on* $A \subset X$*.*

Proof. **(i)** $\Rightarrow$ **(ii):** We look at the special case of (i), when $Y = A$ and $g = id_A$. Then the extension G is the desiered retraction.
(ii) $\Rightarrow$ **(i):** Let $g\colon A \to Y$. Then we define $G\colon X \to Y$ to be the map g precomposed with a retraction $r\colon X \to A$. Since r is the identity on A this indeed is an extension of g. □

To proof, that a space pair actually satisfies the *HEP* it is a convenient way to use the *retraction criterion*. This follows from the previous two lemma. Often, the retraction criterion is only phrased for closed subsets $A \subset X$, but I assumed the clossness condition on A, as in theorem 2.4, only for the implication, where it is needed.

Definition 2.6. Let X be a topological space and $A \subset X$ a subset. A retraction is a continuous map $r : X \to A$, such that precomposition with the inclusion $i : A \hookrightarrow X$ yields the identity on A: $r \circ i : A \to A = id_A$.

Lemma 2.7 (Retraction Criterion)**.** *Let* X *be a topological space and* $A \subseteq X$ *a subspace. Consider the following two statements:*

(i) the pair (X, A) *has the* HEP

(ii) the subspace $X \times \{0\} \cup A \times [0,1] \subseteq X \times [0,1]$ *is a retract of* $X \times [0,1]$

Then (1) $\implies$ (2)*. If, in addition,* A *is closed in* X*, the converse* (2) $\implies$ (1) *holds as well.*

Proof. **(i)** $\Rightarrow$ **(ii):** If (X, A) has the *HEP*, then by "(i) $\implies$ (ii)" from theorem 2.4 we get: for every map $g\colon X \times \{0\} \cup A \times [0,1] \to Y$ there exists an extension of g to a map on $X \times [0,1]$. Now the statement follows directly from theorem 2.4.
(*A* closed $\wedge$ (ii)) $\Rightarrow$ (i): For this direction we first use that $X \cup A \times [0,1] \subseteq X \times [0,1]$ is closed and apply the backward direction of theorem 2.5. This yields the same intermediate step as in the previous prart of this proof / the forward direction. With the same closedness as before, we apply the backward direction of theorem 2.4 and conclude the proof. □

It follows from the retraction criterion, that the *HEP* is preserved under taking the product with a space.

Corollary 2.8. *If* (X, A) *has the* HEP*, so does* $(W \times X, W \times A)$ *for any space* W*.*

Proof. If $r\colon X \times [0,1] \to X \times \{0\} \cup A \times [0,1] \subseteq X \times [0,1]$ is a retract, so is

$$id_W \times r\colon W \times X \times [0,1] \to W \times X \times \{0\} \cup W \times A \times [0,1] \subseteq W \times X \times [0,1].$$

Continuity follows, because id_W is the product of two continuous maps. Furthermore id_W is fixed, where required, since id_W is fixed on all of W. □

–> ToDo: noch in Lean schreiben

Corollary 2.9. *If* (X, A) *has the* HEP *and* J *is a discret set, then* $(J \times X, J \times A)$ *has the* HEP*.*

Proof. Let $r\colon X \times [0,1] \to X \times \{0\} \cup A \times [0,1] \subseteq X \times [0,1]$ be a retract, then also

$$\begin{aligned} id_J \times r\colon J \times X \times [0,1] &\to J \times (X \times \{0\} \cup A \times [0,1]) \\ &= J \times X \times \{0\} \cup J \times A \times [0,1] \end{aligned}$$

is a retract. □

Theorem 2.10. *The* HEP *is preserved under homeomorphisms, i.e. given two space pairs* (X, A)*and*(Y, B)*. If there exists a homeomorphism from* X *to* Y*, that maps* A *to* B*, and* (X, A) *has the* HEP*, then so does* (Y, B)*.*

Proof. Let $\phi\colon X \to Y$, be a homeomorphism, that maps A to B and let $r_X\colon X \times [0,1] \to X \times \{0\} \cup A \times [0,1]$ be a retraction, which we obtain from the Retraction Criterion for the pair (X, A). $pr_X\colon X \times [0,1] \to X$ and $pr_I\colon X \times [0,1] \to X$ are the projections onto the first respectively second component. We use those maps, to define r_Y.

$$\begin{aligned} r_Y\colon Y \times [0,1] &\to Y \times \{0\} \cup B \times [0,1] \\ (y,t) &\mapsto (\phi \circ pr_X \circ r_X(\phi^{-1}(y),t), pr_I \circ r_X(\phi^{-1}(y),t)) \end{aligned}$$

It is left to check, that r_Y is a retraction, which yields the *HEP* for the space pair (Y, B). The map r_Y ...

- is continuous: the two component functions are compositions of continuous maps.
- is fixed on $Y \times \{0\} \cup B \times [0,1]$: Let (y,t) be a point in this subspace of $Y \times [0,1]$. Then, $(\phi^{-1}(y), t)$ is in $X \times \{0\} \cup A \times [0,1]$, because ϕ^{-1} maps Y to X and B to A. So $(\phi^{-1}(y), t)$ lies in exactly the space, which is fixed under r_X. After applying ϕ onto the first component, we observe, that $r_Y(y,t) = (y,t)$.
- has image in $Y \times \{0\} \cup B \times [0,1]$: Let (y,t) be a point in $Y \times [0,1]$. $r_X(\phi^{-1}(y), t)$ is in the image of r_X, which is $X \times \{0\} \cup A \times [0,1]$. It follows from the definition of r_Y, that either the second component of $r_Y(y,t)$ is zero, or the first one lies in B.

□

Theorem 2.11. *Let (A, B) and (B, C) be pairs of topological spaces, that both have the* HEP. *Then also (A, C) has the* HEP.

Proof. We will prove this, by checking the definition of *HEP*. So we are given a topological space Y, a map $f\colon A \to Y$ and a homotopy $H\colon C \times [0,1] \to Y$. Using the *homotopy extention property* of (B, C) for $f|_B$ and H, we obtain a homotopy $H'\colon B \times [0,1] \to Y$, that extends the given maps. Next, we repeat the same with the *HEP* of (A, B) for the maps f and H', to obtain the desired homotopy $H''\colon A \times [0,1] \to Y$. It is obvious, that H'' indeed extends f and H. □

2.2 HEP for relative CW-complexes

To proof the *HEP* for relative CW-complexes, we will generalize the space pair in three steps and proof the *HEP* for each of these pairs. The general strategy is as follows.

1. Any disc relative its boundary has the *HEP*.

2. The pair of two consecutive skeletons has the *HEP*, i.e (X_{n+1}, X_n) has the *HEP* for all n.

3. Any relative CW-comples has the *HEP*.

Before we start proofing the *HEP* for any space pair, I will give some definitinons and introduce notation for CW-complexes.
The follwoing definition of relative CW complexes is a modified version of the definition of CW-complexes from the bachelor thesis of Hannah Scholz [Sch24]. She chose to formalise the historical definition of CW-complexes, presented by Whithead in [Whi18] and explains her work in this thesis. This definition is the basis of her formalisation of CW complexes, which can be found as the "classical CW-complex" in mathlib. I want to present this definition here, because it has all the structures, I used in the my formalised proofs.

Definition 2.12. We call $D^n := \{x \in \mathbb{R}^n \mid \|x\| \leq 1\}$ $n-$dimensional closed ball. Its boundary $\partial D^n = S^{n-1} := \{x \in \mathbb{R}^n \mid \|x\| = 1\}$ is the $(n-1)-$dimensional sphere.

Definition 2.13. Let X be a Hausdorff space. A *relative CW-complex* on (X, C) consists of a family of indexing sets $(I_n)_{n\in\mathbb{N}}$ and a family of maps $(Q_i^n \colon D_i^n \to X)_{n\geq 0, i\in I_n}$ s.t.

1. $Q_i^n|_{\mathring{D}_i^n} : \mathring{D}_i^n \to Q_i^n(\mathring{D}_i^n)$ is a homeomorphism. We call $e_i^n := Q_i^n(\mathring{D}_i^n)$ an *(open) n-cell* (or a cell of dimension n) and $\overline{e}_i^n := Q_i^n(D_i^n)$ a *closed n-cell*.

2. For all $n, m \in \mathbb{N}$, $i \in I_n$ and $j \in I_m$ where $(n, i) \neq (m, j)$ the cells e_i^n and e_j^m are disjoint.

3. For each $n \in \mathbb{N}$, $i \in I_n$, $Q_i^n(\partial D_i^n)$ is contained in the union of C and a finite number of closed cells of dimension less than n.

4. $A \subseteq X$ is closed iff $C \cap A$ is closed and $Q_i^n(D_i^n) \cap A$ is closed for all $n \in \mathbb{N}$ and $i \in I_n$.

5. $X \cup \bigcup_{n\geq 0} \bigcup_{i\in I_n} Q_i^n(D_i^n) = X$.

We call Q_i^n a *characteristic map* and $\partial e_i^n := Q_i^n(\partial D_i^n)$ the *frontier of the n-cell* for any i and n. Additionally we define $X_n := \bigcup_{m<n+1} \bigcup_{i\in I_m} \overline{e}_i^m$ and call it the *n-skeleton* of X for $-1 \leq n \leq \infty$

Definition 2.14. If $C = \emptyset$, we call $(X, \emptyset)$ or just X, an (absolute) CW complex.

Definition 2.15. A relative CW complex (X, A) is called finite dimensional, if there exists a $n \in n$, such that $I_m = \emptyset$ for all $m \geq n$.

2.2.1 HEP for balls

Theorem 2.16. $\forall n \in \mathbb{N}$: $(D^m, \partial D^m)$ *has the* HEP.

There are two different attempts to prove this theorem. One is quite visual via the retraction criterion and the other one constructs the extended homotopy explicitly. Here, I will give the constructive proof, because that one was easier to formalize in Lean.

Proof. Given a homotopy $H : \partial D^m \times [0,1] \to Y$ and a map $f\colon D^m \to Y$, which agree on $\partial D^m \times \{0\}$ identifyed with ∂D^m. We assemble these to a map $h\colon D_2^m \to Y$, where D_2^m denotes a disc with radius 2 in $\mathbb{R}^m$. h is defined in the following way.

$$h(x) = \begin{cases} f(x) & \text{if } \|x\| \leq 1 \\ H(x/\|x\|, \|x\| - 1) & \text{else} \end{cases}$$

Using h we define the desired homotopy $H'\colon D^m \times [0,1] \to Y$ to be

$$H'(x,t) = h((1+t)x)$$

It is left to check that this actually defines a continuous extension of the homotopy H and that it starts with f.
–> wenn zu kurz, dann hier ausführlicher □

Since the definition of CW-complexes in Mathlib uses n-dimenstional cubes instead of closed balls as preimage of a closed cell under the caracteristic map, we will state the *HEP* for cubes relative to their boundary.

Corollary 2.17. $\forall n \in \mathbb{N} : ([-1,1]^n, \partial[-1,1]^n)$ *has the* HEP.

Proof. This statement follows from the observation, that D^m is homeomorphic to $[-1,1]^n$. One constructs a homeomorphism via the radial projection, which maps the boundary of the sphere to the boundary of the cube. All points in the interior are scaled along the rays. In that way, the cube inherits the *HEP* from the sphere. □

2.2.2 HEP for consequitive skeletons

In the next step, we will prove, that for every relative CW complex (X, C) with skeleton X_n and for all $n \in \mathbb{N}$, the pair (X_{n+1}, X_n) has the *HEP*. We prove this by applying the retraction criterion, which means we construct a retraction $X_{n+1} \times [0,1] \to X_{n+1} \times \{0\} \cup X_n \times [0,1]$. This map will be obtained from the universal property of the quotient. Tho show, that $X_{n+1} \times [0,1]$ is a quotient space, it suffices to prove, that X_{n+1} is a quotient space, and then apply the follwoing theorem:

Theorem 2.18. *For every locally compact space X and any quotient mapping $g\colon Y \to Z$, the Cartesian product*

$$g \times id_X \colon Y \times X \to Z \times X$$

is a quotient mapping.

So let us prove, that every skeleton indeed is a quotient space: We set $Z_m := C \amalg \coprod_{n\leq m} \coprod_{i\in J_n} D_i^n$ and define the map

$$\begin{aligned} P_m : Z_m &\to X_m \\ x &\mapsto \begin{cases} Q_i^n(x), & \text{if } x \in D_i^n, \\ x, & \text{else } (x \in C) \end{cases} \end{aligned}$$

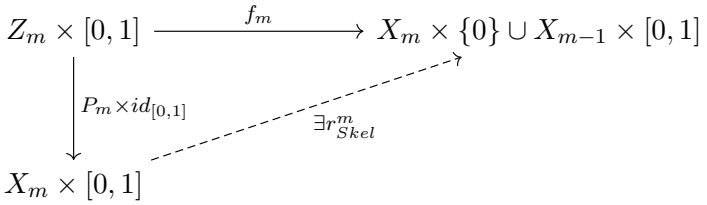
Lemma 2.19. *Let (X, C) be a relative CW-complex with skeleton $X_m, m \in \mathbb{N}$. The map $P_m : Z_m \to X_m$ is a quotient map for every $m \in \mathbb{N}$.*

Proof. The map P_m is well defined, since the image of the characterstic maps up to dimension m is in X_m and the base space C is contained in every skeleton. Surjectivity is clear from the construction of CW complexes. It is left to show, that a set of $S \subset X_m$ is closed iff $P_m^{-1}(S)$ is closed. The forward direction we get from the map P_m being continuous (defined in terms of continuous maps on a disjoint union). For the reverse direction we have, that $P_m^{-1}(S)$ intersected with the base space and every closed ball is closed. The $m-$skeleton only consists out of the base space and cells up to dimension m. So to prove closedness of a subset of the $m-$skeleton, it suffices to show, that the intersection with the base space and with all closed cells up to dimension m is closed. This follows from the observation, that $P_m^{-1}(S) \cap D_i^n = S \cap \overline{e}_i^n$ and $P_m^{-1}(S) \cap C = S \cap C$. □

Now, that we know $P_m \times id_{[0,1]} \colon Z_m \times [0,1] \to X_m \times [0,1]$ is a quotient map, we proceed by defining the map

$$\begin{aligned} f_m : Z_m \times [0,1] &\to X_m \times \{0\} \cup X_{m-1} \times [0,1] \\ x &\mapsto \begin{cases} (P_m \times id_{[0,1]}) \circ r_i^m(x), & \text{if } x.1 \in D_i^m, i \in I_m \\ (P_m \times id_{[0,1]})(x), & \text{else} \end{cases} \end{aligned}$$

. Here $r_i^m \colon D_i^m \times [0,1] \to D_i^m \times \{0\} \cup S_i^{m-1} \times [0,1]$ is the retraction, whose existens we get from the retraction criterion and theorem 2.16.
If we show, that f_m is continuous and factors through $P_m \times id_{[0,1]}$, then we obtain the retraction r_{Skel}^m for consecutive seketons from the universal property of the quotient, as illustrated above. Of course it is a priory not clear, that the obtained map actualy is the desired retraction. This will be checked later on.



Lemma 2.20. *The map $f_m, m \in \mathbb{N}$*

(i) is continuous

(ii) is well defined

(iii) factors through $P_m \times id_{[0,1]}$

Proof.
(i) : The map f_m is defined on the domain $Z_m \times [0,1]$, which is is homeomorphic to $(C \times [0,1]) \amalg \coprod_{n \leq m} \coprod_{i \in J_n} (D_i^n \times [0,1])$, via the natural mapping. Continuity of f precomposed with the homeomorphism is easy to show, since is the product and composition of continuous maps on the disjoint union. From the homeomorphisms we get, that f as defined earlier is continuous.
(ii): To check well definedness, we consider two different cases, same as in the definition of f_m. First let $(x,t) \in D_i^m \times [0,1]$ for some $i \in I_m$, so we are the "if"- case of the definition. Since r_i^m is a retraction, we get the following result.

$$\begin{aligned} f_m(x,t) &= (P_m \times id_{[0,1]})(r_i^m(x,t)) \\ &\subset (P_m \times id_{[0,1]})(D_i^m \times \{0\} \cup S_i^{m-1} \times [0,1]) \end{aligned}$$

Now, $(P_m \times id_{[0,1]})(r_i^m(x,t))$ will either have the second coordinate to be zero, and hence lies in $X_m \times \{0\}$, or the first coordinate lies in the boundary, which is glued to the $m-1-$skeleton, and hence lies in $X_{m-1} \times [0,1]$.
The "else-"case is even more obvious. If $(x,t) \in Z_m \times [0,1]$ is a point, where x lies in a closed ball, that has dimension less than m, then $P_m \times id_{[0,1]}$ will map this point into $X_{m-1} \times [0,1]$.
(iii): Assume we have two points (z,t) and $(z',t') \in Z_m \times [0,1]$, such that $P_m \times id_{[0,1]}(z,t) = P_m \times id_{[0,1]}(z',t')$. $P_m \times id_{[0,1]}$ is the indentity on the second coordinate, so we directly get $t = t'$ and $P_m(z) = P_m(z')$. To show, that f_m factors through $P_m \times id_{[0,1]}$, we have to show, that $f_m(z,t) = f_m(z',t)$.
For most points, applying f_m is definitionally equal to applying $P_m \times id_{[0,1]}$. Only if at least one of z and z' is in a top dimensional ball, something could go wrong. To be precise, something could only go wrong, when at least one of them is in the interior of a top dimensional cell, since on the boundary, theretraction r_i^m is the identity and hence f_m again is the same as $P_m \times id_{[0,1]}$. So what we want to show is, that if $z \in \mathring{D}_i^m$, then $z = z'$.
$P_m(z) = Q_i^m(x)$ and from the definition of a relative CW complex we have,

that $Q_i^m|_{\mathring{D}_i^m} : \mathring{D}_i^m \to e_i^m$ is a homeomorphism. Hence, there is no other point $x \neq z$ in $\mathring{D}_i^m$, s.th. $P_m(x) = P_m(z)$, which means, that our goal reduces, to showing, that z' lies in the same open ball as z. If z' was in a different open ball $\mathring{D}_j^m, j \neq i$ than $z \in e_i^m$ and $z' \in e_j^m$. Open cells of the same dimension are disjoint, which contradicts $P_m(z) = P_m(z')$. If z' was in the boundary of an open ball of dimension m, or in any closed ball of dimension less than m, then $P_m(z')$ lies in the $(m-1)-$skeleton, which is also disjoint from e_i^m. This again would contradict $P_m(z) = P_m(z')$.

□

Theorem 2.21. *Let (X, C) be a relative CW-complex with skeleton $(X_m)_{m\in\mathbb{N}}$, then $\forall m \in \mathbb{N} : (X_m, X_{m-1})$ has the* HEP.

Proof. As discussed earlier, we have the map $r_{Skel}^m \colon X_m \times [0,1] \to X_m \times \{0\} \cup X_{m-1} \times [0,1]$ from the above diagram. We get continuity from the universal property and we know from the construction and theorem 2.20, that it maps into the right space. Therefore it is only left to show, that precomposition with the inclusion $\iota \colon X_m \times \{0\} \cup X_{m-1} \times [0,1] \hookrightarrow X_m \times [0,1]$ is the identity. If we have a point $(x,t) \in X_{m-1} \times [0,1]$, there is not necessarily a unique choice for a preimage in $Z_m \times [0,1]$. But since we proved in theorem 2.20, that f_m factors through $P_m \times id_{[0,1]}$, we know, that $r_{Skel}^m(x,t)$ is indemendet of this choice. That means we can chose a preimage $(z,t) \in Z_m \times [0,1]$ of (x,t), that lies <u>not</u> in a closed ball of dimension m. Therefore, by the definition of f_m, we have

$$f_m(z,t) = (P_m \times id_{[0,1]}(z,t)) = (x,t)$$

Together with the equality from the comutative diagram, we get that

$$\begin{aligned}(x,t) = f_m(z,t) &= r_{m+1}^{Skel}(P_{m+1} \times id_{[0,1]}(z,t)) \\ &= r_{m+1}^{Skel}(x,t)\end{aligned}$$

So r_{Skel}^m is the idenity on those type of points.
Now we consider the other case, that the point $(x,0)$ lies in $(X_{m+1}) \times \{0\}$. By the same argumenation as before, we can choose any preimage $(z,0)$ in $D_i^m \times \{0\}$ for some $i \in I_m$, such that $P_m \times id_{[0,1]}(z,0) = (x,0)$, and use this for the computations. Therefor $f_m(z,0) = (P_m \times id_{[0,1]}) \circ r_i^m(z,0)$. Now r_i^m is a retraction and hence fixed on $D_i^m \times \{0\}$ $subset D_i^m \times \{0\} \cup S_i^{m-1} \times [0,1]$. This yields the following equalities.

$$\begin{aligned}f_m(z,0) &= (P_m \times id_{[0,1]}) \circ r_i^m(z,0) \\ &= (P_m \times id_{[0,1]})(z,0) \\ &= (x,0)\end{aligned}$$

Again, we compare this with the composition from the comutative diagram

$$\begin{aligned}(x,0) = f_m(z,0) &= r_{Skel}^m(P_m \times id_{[0,1]}(z,0)) \\ &= r_{Skel}^m(x,0)\end{aligned}$$

and conclude, that also for those points r^m_{Skel} is the identity map. □

2.2.3 HEP for relative CW complexes

In this last section, we will prove, that every relative CW complex (X, C) has the *HEP*. It is easy tp prove this for finite dimensional CW complexes:

Lemma 2.22. *Let (X, C) be a relative CW complex. Then (X_n, C) has the* HEP *for all $n \in \mathbb{N} \cup \{-1\}$.*

Proof. The claim follows by induction on n.
Induction start $n = -1$: Note, that $X_{-1} = C$, i.e. $(X_{-1}, C) = (C, C)$. This pair has the *HEP* by .
Induction step $n \mapsto (n+1)$: Assume, we know that (X_n, C) has the *HEP*. From theorem 2.21 we know, that also X_{n+1}, X_n has the *HEP*. So theorem 2.11 yields, that also (X_{n+1}, X_n) has the *HEP* □

Corollary 2.23. *Let (X, C) be a finite dimensional relative CW complex. Then (X, C) has the* HEP.

Proof. Since (X, C) is finite dimensional, there exists a $n \in \mathbb{N} \cup \{-1\}$, such that X is equal to the $n-$ skeleton X_n. So we have an equality of the pairs $(X, C) = (X_n, C)$. The claim follwos from theorem 2.22. where we proved the *HEP* for the right handside. □

I stoped with the formalisation at this point, nevertheless, I want to roughly present the prove of the general version here, for the sake of completness.

Lemma 2.24. *Let (X, C) be a relative CW complex. Then (X, C) has the* HEP.

Proof. We prove this by the retraction criterion. From theorem 2.22 and the retraction criterion we have retractions $r_m \colon X_m \times [0,1] \to X_m \times \{0\} \cup C \times [0,1]$ for all $m \in \mathbb{N}$. I leave it to the reader to check, that r_m can be explicity constructed as the composite

$$X_m \times [0,1] \xrightarrow{r^{Skel}_m} X_m \times \{0\} \cup X_{m-1} \times [0,1] \xrightarrow{id_{X_m \times \{0\}} \cup r_{m-1}} X_m \times \{0\} \cup C \times [0,1]$$

and to observe, that from this construction we get $r_m|_{X_{m-1} \times [0,1]} = r_{m-1}$.

We define $r \colon X \times [0,1] \to X \times \{0\} \cup C \times [0,1]$, as $r(x,t) = r_m(x,t)$, if $x \in X_m$. This m is not unique, nevertehless, r is well defined, by the above observation. It is left to check, that r is well defined and fixed on $X \times \{0\} \cup C \times [0,1]$. Continuity follows from the following proposition, which I now just want to claim :

Proposition 2.25. *Let (X, C) be a relative CW complex and Z a locally compact space. Then the map $f\colon X \times Z \to Y$ is continuous if and only if for all $n \geq -1$, the restriction $f|_{X_n \times Z}$ is continuous.*

□

3 Lean and mathlib

After taking a closer look at the informal mathematics behind my formalisation in the previous chapter, I now want to give a brief overview of what Lean is. I will explain some of the conventions and concepts used in formalization that are relevant to this project.
Lean is a proof assistant, i.e. a tool that checks and confirms the individual reasoning steps of mathematical proofs written in Lean's formal language. It verifies that assumptions, axioms, functions, and theorems are only applied where permitted, and that new results are derived using valid logical inferences. In *The Lean Theorem Prover (system description)* the main designers of Lean explain that the correct verification of mathematical proofs in Lean rests upon a "small trusted kernel based on dependet type theory" (leanTP). I will elaborate the term "dependet type theroy" later in this section.
Lean is one of several proof assistants, alongside others such as Rocq or Isabelle. However, as Alex Kontorovich elaborates in *The Shape of Math to Come* [Kon25], it is remarkable how quickly Lean is gaining relevance, not only in computer science and mathematical logic, but also in mainstream mathematical research.
Since July 2023, the non-profit *Lean Focused Research Organization (FRO)* has supported and coordinated the ongoing development of the language. Its stated goal is "enhanced scalability, usability, documentation, and proof automation while guiding Lean toward long-term self-sustainability" [FRO]. The same source also provides a brief overview of Lean's development history. The first version was created by *Leonardo de Moura* in 2013. Since then, the language itself has evolved. The first stable release of Lean 4 appeared in September 2023, and both the community and the digital library *mathlib* have grown considerably. Today, mathlib 4 contains more than 2,000,000 lines of code and has over 770 contributors. Current statistics can be found at [].

3.1 Lean's Dependent Type Theory

The digital book *Theorem Proving in Lean 4* teaches how to verify proofs in Lean and explains the underlying logical system. Its second chapter, on *Dependent Type Theory*, serves as the basis for the following description. The first and probably most important fact is that every expression in Lean has a type. Using the command `#check`, one
can display the type of an expression. In the example below, I first define `n` to be the natural number `3` and then check its type, which is $\mathbb{N}$ by the definition above. As stated, every expression in Lean has a type, so the natural numbers themselves can be checked aswell. Their type is called `Type`. The classical objects $\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$, as well as the finite-dimensional Euclidean spaces, all have type `Type`. Continuing the same procedure and checking the type of `Type`

yields `Type : Type 1`. This can be repeated indefinitely: in general, `Type k` has type `Type (k+1)` for every natural number `k`. I want to present one more Type, which is called `Prop`. As the name already implies, Prop is the type used for propsoitions, e.g. `2 = 1` . We can combine propositions two build new ones. For example we can connect two propositions with a logical "and" in order to obtain a new expression of type `Prop`. `Prop` itself has type `Type`.

```
def n : ℕ := 3
#check n
     n :  ℕ
#check ℕ
     ℕ :  Type
#check EuclideanSpace ℝ (Fin 3)
     EuclideanSpace ℝ (Fin 3) :  Type
#check Prop
     Prop :  Type
#check Type
     Type :  Type 1
#check 2 = 1
     2 = 1 :  Prop
#check Prop
     Prop :  Type
```

Type theory not only allows one to define an element, such as the natural number `3` above, but also to construct new types from existing ones. Functions are one such example. To illustrate this, I start by choosing two arbitrary but fixed types α and β. Writing `Type*` admits types of any universe. Lean then automatically introduces a so-called universe variable, here `u_1`. A function with domain α and codomain β has type $\alpha \to \beta$. This newly constructed type again has a type of its own, whose universe is the maximum of the universe levels of α and β.

```
variable (α β : Type*) (f : α → β)
#check α
     α :  Type u_1
#check f
     f :  α → β
#check α → β
     α → β :  Type (max u_1 u_2)
```

Functions such as the addition of natural numbers, which one would usually write as a map $\mathbb{N} \times \mathbb{N} \to \mathbb{N}$, have in Lean the type $\mathbb{N} \to \mathbb{N} \to \mathbb{N}$. This means that after applying a single natural number to the addition function, we obtain a function $\mathbb{N} \to \mathbb{N}$ that adds the first number. Since $\mathbb{N}$ has type `Type`, the reasoning of the previous paragraph shows that the function type $\mathbb{N} \to \mathbb{N} \to \mathbb{N}$ again has type `Type`. Note the distinction between the two levels: the addition function itself has type $\mathbb{N} \to \mathbb{N} \to \mathbb{N}$, whereas this function type in turn has type `Type`.

```
#check Nat.add
     Nat.add : ℕ → ℕ → ℕ
#check ℕ → ℕ → ℕ
     ℕ → ℕ → ℕ : Type
```

Even though I do not want to illustrate in detail, what makes the type theory of lean a dependet type theory, let me

3.2 Structures

3.3 Junk Values

If you write a division by zero in a formalized theorem, Lean returns the result 0.

```
example : 1 / 0 = 0 := by norm_num
```

This makes division in Lean always well-defined, even though certain inputs have no genuine mathematical meaning. As explained in [MLS25], the use of so-called *junk values* is common practice. The same convention applies, for instance, to the inversion of square matrices, which is always defined and returns the zero matrix whenever the matrix is singular, or to the derivative of a non-differentiable function, which is defined to be zero. Many further examples of this kind exist. At first glance this may seem concerning, since such values have no mathematical justification and could appear to enable incorrect proofs. Indeed, one can state and prove propositions that, in classical mathematics, would look false or ill-defined. This issue is resolved by adding suitable hypotheses such as `n≠ 0` or `Differentiable k f` to any statement about the object in question. They ensure we only use the result of the theorem in apllicable situations, such as, if our number is nonzero or our function is differentiable. Consequently, although the convention may seem alarming initially, relying on junk values is standard practice in Lean formalizations, precisely because it simplifies both definitions and statements. Returning to the first example, it is what allows division to be expressed as a total map $\mathbb{Q} \to \mathbb{Q} \to \mathbb{Q}$.

3.4 API lemma

4 Lean formalisation of the Homotopy Extension Property

4.1 CW-complexes in Mathlib

1. NOTIZEN:
2. Cellen im CW complex sind würfel anstatt spheres
3. CW -complexe leben in einem größeren Raum ->

4.2 HEP Definition

Already in the mathematical section, the definition of the *homotopy extension propert* was quite long and technical. The same is true for its formalization. Currently I have three versions of the definition in my project, since we decided within the work progress, that different phrasings might be more applicable. I will later on discuss the pros and cons for each of the version. The first definition is the following, called `HEP`:

```
def agreeOn.{u} {X Y : Type u} (f : X → Y) (H : X × ℝ → Y) (A :
    Set X) : Prop :=
    ∀ (a : X), a ∈ A →  f a = H (a, 0)

structure HomotopyExtension.{u} {X Y : Type u} [TopologicalSpace X]
    [TopologicalSpace Y] (H' : X × ℝ → Y) (f : X → Y)
  (H : X × ℝ → Y) (A : Set X) (rangeH' : Set Y) : Prop where
  ContinuousOn : ContinuousOn H' { p : X × ℝ | p.2 ∈ unitInterval}
  range : { p : X × ℝ | p.2 ∈ unitInterval}.MapsTo H' rangeH'
  agreef : (∀ (x : X), f x = H' (x, 0))
  agreeH : (∀ (a : A) (t : unitInterval), H (a,t) = H' (a, t))

def HEP.{u} (X : Type u) [TopologicalSpace X] (A : Set X) : Prop :=
  ∀ (Y : Type u) [TopologicalSpace Y] (rangeH' : Set Y), ∀ (f : X →
     Y), ∀ (H : X × ℝ → Y),
  Continuous f → Set.range f ⊆ rangeH' →
  ContinuousOn H {p : X × ℝ | p.1 ∈ A ∧ p.2 ∈ unitInterval} →
  {p : X × ℝ | p.1 ∈ A ∧ p.2 ∈ unitInterval}.MapsTo H rangeH' →
    agreeOn f H A
  → ∃ H' : X × ℝ → Y, HomotopyExtension H' f H A rangeH'
```

To shorten the definition a little and in order to make it more readable, I first defined `agreeOn`. This tells, that the function `f` and the homotopy `H`, which we want to extend, have the same values on the subset `A` respectively `A × 0`. So it ensures, that the two maps are compatible and it makes sence to ask for an extension. Secondly I constructed the structure called `HomotopyExtension`, which combines all the properties, our desired homotopy should have. Namely, continuity, that its image lies in the right space and

that it agrees with the two functions given in the begin. Making this a structure, instead of a simple definition has the advantage, that it comes with the projection maps for each property.
If we now take a closer look at the definition of the homotopy extension property, it is pretty similar to the mathematical definition theorem 2.1 of the *HEP*. Still, there is one point, I want to discuss. Defining the homotopies as a map on `X × ℝ` instead of `X × unitInterval` or `A × unitInterval`, makes it easier to work with it. For instance consider the definition `agreeOn`. Assume, the homotopy `H` was defined as a map on `A × unitInterval`, then one had to provide a proof of `a ∈ A`, every time one wants to apply a point `(a : X)` to the function. Also in the second argument, we can apply any real number to it, without the need to provide a prove that this real number lies between zero and one.
Still, this does not change anything about the defined object. `ContinuousOn H ' { p : X × ℝ | p.2 ∈ unitInterval}` and `∀ (a : A) (t : unitInterval ), H (a,t) = H' (a, t)` make it irrelevant, what happens outside of the compact interval, but instead put precise focus on what we need for our definition.
Moreover, the set `rangeH'` has no equivalent in the informal definition, but is a result of formalisation. `rangeH'` is a set of the codomain-type `Y` and as the name indicates, the image of the extendet homotopy `H'` is contained in it. Together with adding this set to the definition, there are two hypotheseses, which we need to add. `H'` cannot have image in `rangeH'`, if not `f` and `H` already have image in there.
The win of adding `rangeH'` to the definition is again, that we can avoid working with subtypes. If we want to construct an extended homotopy, which has image in a subtype, we now get a map between types together with the result, that the image lies in the required subset. Neverthless, in most cases, one wants to choose `rangeH'` to be all of Y. In Lean this can be expressed as `@ Set.univ Y`. The two additional assumptions are then statements, which can be easily prooven by `tauto`. Since this extra work is very manageable, I prefere this definition of `HEP` over the following version called `HEPY`, where `rangeH'` is set to be the entire codomain `Y`. Even tough this one is shorter and closer to the informal version. The `leamm HEP_iff_HEPY` proofs, that those two definitions are equivalent

```
structure HomotopyExtensionY.{u} {X Y : Type u} [TopologicalSpace X]
     [TopologicalSpace Y] (H' : X × ℝ → Y) (f : X → Y)
  (H : X × ℝ → Y) (A : Set X) : Prop where
  ContinuousOn : ContinuousOn H' { p : X × ℝ | p.2 ∈ unitInterval}
  agreef : ∀ (x : X), f x = H' (x, 0)
  agreeH : ∀ (a : A) (t : unitInterval), H (a,t) = H' (a, t)

def HEPY (X : Type u) [TopologicalSpace X] (A : Set X) : Prop :=
  ∀ (Y : Type u) [TopologicalSpace Y], ∀ (f : X → Y), ∀ (H : X × ℝ
```

```
    → Y), Continuous f →
  ContinuousOn H {p : X × ℝ | (p.1 ∈ A) ∧ (p.2 ∈ (unitInterval))}
    → agreeOn f H A
  → ∃ (H' : X × ℝ → Y), HomotopyExtensionY H' f H A
```

The last aspect, I want to elucidate, before progressing to the final third version of the definition is, that I fixed the Type of `X` and `Y` to be the same, namely bothe are `Type u`, for some arbitrary universe `u`. This was needed, in order to .... – ToDo
Let us now compare this definition with the later version. `HEP'` :

```
open Set.Notation
def HEP' {Y : Type*} [TopologicalSpace Y] (X A : Set Y) : Prop :=
    HEP X (X ↓∩ A)
```

The notation `(X ↓∩ A)` means, consider `(X ∩ A)` as a set of Type `Set X`. Then `HEP'`, is defined in terms of `HEP` for the pair `X` and `(X ↓∩ A)`. The difference to the previous versions of the definition is, that in the definition `HEP`, `A` is a subset of the type `X`. That means that the pair of spaces does not live in an ambient space. Wherelse, in the definition `HEP'`, both sets X and A are sets of the same type `Y`.
We decided to go for the second version of the definition, when we wanted to phrase the *homotopy extension property* for pairs of consecutive skeletons. A skeleton of a relative CW complex in Lean is a subcomplex, whose carrier is a subset of an ambient space. Also it turned out to be the nicer way, to prove lemmas about closed balls and spheres, because I do not have to concider the sphere as a subset of "Type closedBall" anymore, but just as the points with distance one to the origin in a metric space.
Note, that in this definition the functions `f` and `H`, which we want to extend are defined on the subtype ↑`X`, respectively ↑`X`×ℝ. Working with those subtypes caused some extra work in the proof of `retraction_criterion_closed'` and also `HEP_trans'`. One could avoid this by modifying the definition `HEP'` in a away, that it is not defined unsing `HEP`, but constructed explicitly with functions defined on `Y`, respectively `Y`×ℝ. The definition in that case would be longer, since for example continuity of f would cange to a `ContinuousOn`-statement. Also, it would be more work to proove equivalence between `HEP` and this newer version, that it was now, where we basically got it for free. Nevertheleess I am convinced, it would impoove the formalisation of the definition.
To sum up, I value the primed version `HEP'` to be the most usfull one to work with. Optimally with the discussed change, to write the definition explicitly with functions defined on the entire space instead of subspaces. Still, I dont think this outdoes the definition `HEP`, in all accounts. Since the statements become longer and also the proof of the retraction criterion would have been longer and more tedious.

4.3 Retraction Criterion

Before elucidating the retraction criterion its formalisation, we look at the definition of a retraction. The classical way to define a retraction r from B to A is, as in theorem 2.6, via precomposition with the inclusion $A \hookrightarrow B$. As already in the formalisation of the definition *HEP*, we again want to avoid working with subtypes. So insetad of defining a map from B to A, for the formalisation, the retraction map is defined from an ambient type X to itself. The structure `RetractionOn` bundles all properties, which make this map a retraction.

```
structure RetractionOn.{u_1} {X : Type u_1} [TopologicalSpace X] (r :
    X → X) (B A : Set X) : Prop where
  subset : A ⊆ B
  continuousOn : ContinuousOn r B
  mapsTo : B.MapsTo r A
  fixesOn : ∀ a ∈ A, r a = a
```

As mentioned before, that the map `r` is a map from a Type X into itself and it is not a retraction on all of its domain. As the name `RetractionOn` implies, it just "retracts" a subset `B` to a subset `A` of the `Type X`. The first property `subset` states, that `A` is a subset of `B`. Note, that leaving out this property in our structure would define a more general class of selfmaps, which does not necessarily sattisfy $r(B) = A$ anymore. The property `continuousOn` ensures, it is a continuous Map on the relevant subset B. The codomain of the retraction is supposed to be A, so the property `mapsTo` claims, that `r` maps all elements of `B` to an element in `A`. Furthermore the classical definition states, that precomposition with the inclusion $A \hookrightarrow B$ should give the identity id_A. In our case there is not really an inclusion map, since both - image and preimage of `r` - are the same `Type X`. So in our setting this condition translates to, that each element in A needs to be fixed by applying r. This property is called `fixedOn`.

Recall, that the retraction criterion was a corollary of two lemmas. The same structure can be found in the formalisation. theorem 2.4 has two implications, one of which requires closedness. In order to only assume closedness when needed, in my project each implication has its own lemma, called `if_HEP_then_extension` and `if_extension_then_HEP`. The formalised proofs are quite unspectacular and just follow the pattern of the informal proof found in the above section/chapter?. The same is true for the second lemma theorem 2.5 as well as the retraction criterion theorem 2.7 itself.

```
lemma retraction_criterion_closed.{u} {X : Type u} [TopologicalSpace
    X] {A : Set X} (hA1 : IsClosed A) :
  HEP X A ↔ ∃ r : X × ℝ → X × ℝ, RetractionOn r {p | p.2 ∈
    unitInterval} {p | p.2 = 0 ∨ p.1 ∈ A ∧ p.2 ∈ unitInterval}
```

The retraction in this above lemma is a map on the product `X × ℝ`, where X is the bigger space of the space pair from the homotopy extension property. Even tough this lemma is phrased for `HEP`, we can also apply it to `HEP'`:

```
variable {Y : Type u} [TopologicalSpace Y] (X A : Set Y) (h : HEP' X
    A)
#check if_HEP_then_retraction h
```

Even tough this yields a retraction for the relevant spaces, this does not quite assemble the idea of `HEP'`. Instead I formalised the lemma `retraction_criterion_closed'`, where the retraction is a map from the ambient space $\times\mathbb{R}$ to itself. This is realised in the following lemma. In the lean file you can again find the forward implication without closedness assumption.

```
lemma retraction_criterion_closed'.{u} {Y : Type u} [
    TopologicalSpace Y] (A X : Set Y) (hAX : A ⊆ X) (hA1 : IsClosed
    A) :
  HEP' X A ↔ ∃ s : Y × ℝ → Y × ℝ, RetractionOn s {p | p.1 ∈ X ∧
    p.2 ∈ unitInterval} {p | p.1 ∈ X ∧ p.2 = 0 ∨ p.1 ∈ A ∧ p.2 ∈
    unitInterval}
```

I proved the `retraction_criterion_closed'` using the `retraction_criterion_closed`. Explicitly, for the forward direction, I obtained a retraction `r` as in ToDo. In order to extend this map `r` to a map `s : Y × ℝ → Y × ℝ`, I precomposed in the first component with a map called `mapYX : Y → X`, which is the identity on the set `X` and maps all other points of Y to an arbitrary junk value in the unitInterval. `Subtype.val` sends `(r x).1 ∈ X`, to the corresponding point in Y. Afterwards, I had to confirm this new defined map `r` indeed is a retraction, i.e. it is continuous on the relevant set, maps to the required one and the necessary values remain unchanged under applying `r`.
For the backward direction I start with a retraction `s : Y × ℝ → Y × ℝ` and want to construct `r :↑X × ℝ →↑X × ℝ`, so applying `retractioncriterionclosed` yields, the claim. One might expect this direction to be easier, since we dont have to construct any additional values, but just restrict `s`, to the desired space. Sadly these dreams shatter at the point, where we want the image to lie in $X \times I \subset Y \times \mathbb{R}$. `s` is a retraction on , so the `RetractionOn.mapsTo` property gives no information about, where in $Y \times \mathbb{R}$ we are, if the second coordinate comes from outside the unit interval. I solved this issue by procomposing the real coordinate with the map `c : ℝ → ℝ := fun t ↦ (Set.projIcc (0 : ℝ) 1 zero_le_one t)`. `Set.projIcc`, is a continuous map from mathlib, which retracts a linear ordered Type to a closed interval in it. This precomoposition makes, that for every point, which we apply to `s`, the `mapsTo` property of `s` is applicable. This extra work, is a downside of working with types instead of subtypes. It would have been avoided, if I defined a homotopy only on `Y × unitInterval`, instead of `Y × ℝ`.
I want to point out, that even tough the two versions of the retraction

criterion are phrased with two different versions of the definition, this was more a choise based to style, than on technical need. Also, I experienced both of these versions to be valuable. While it might be advisable to just choose the definition `HEP'`, I see usage in both versions of the retraction criterion. The usage of `retractioncriterionclosed` is underlined by the the proof, that the pair of two consequitive skeletons satisfies the *HEP*. I constructed a retraction `r: (skeletonLT X m)` $\times \mathbb{R} \to$ `(skeletonLT X m)` $\times \mathbb{R}$. The key theorem for the construction of this continuous map was the lemma `Topology.IsQuotientMap.continuous_lift_prod_left`, which is a formalisation of Whitehead's Theorem about products of quotient maps with a locally compact factor. If I worked with ambient spaces, I would have lost the surjectivity of the quotient map and would not have been able to apply this therem, the way it can be found in mathlib.

4.4 Preservation of the HEP

Besides the retraction criterion I formalised two more theorems, which are usefull to prove, that a pair of topological spaces has the *HEP*. One is, that the *HEP* is preserved under homeomorphisms. The other is sort of a transitivity property, i.e. if $(A1, A2)$ and $(A2, A2)$ are pairs of topological spaces and both of them satisfy the *HEP*, then also the pair $(A1, A3)$ has the *HEP*.

```
lemma HEP_trans.{u_1} {X : Type u_1} [TopologicalSpace X] (A1 A2 A3 :
    Set X) (h21_sub : A2 ⊆ A1) (h12_hep : HEP' A1 A2) (h32_sub : A3
    ⊆ A2) (h23_hep : HEP' A2 A3) : HEP' A1 A3
```

For the formalisation of the proof, I had the choice between two different attempts. Instead of applying the retraction criterion to all the `HEP` statements, and then using the two retractions from my assumptions to construct the desired retraction, I proved the lemma by checking the definition.
Another positive side effect of this choise was, that I don't have to assume closedness of any Sets, which would be required to apply the retraction criterion.

4.5 HEP for discs and cubes

The goal of the file `HEP_ball_cube.lean` is, to prove the homotopy extension property for balls and cubes relative to their boundary. These are not just the first interesting space pairs, we want to prove the *HEP* for, but also the first step in the prove of the *homotopy extension property* for relative CW complexes.
We start with the balls: As in the prove of theorem 2.16, I first constructed the auxilliar functionion `auxfun` combining the information of the given maps `f` and `H`. Then this function is used to explicitly define a desired homotopy `H'`, which we need to check the definition fo `HEP`

```
variable {m : ℕ} {X Y : Type} [TopologicalSpace X] (C : Set Y)
  (f : (Metric.closedBall (0 : EuclideanSpace ℝ (Fin m)) 1) → Y)
  (H : (Metric.closedBall (0 : EuclideanSpace ℝ (Fin m)) 1) × ℝ →
    Y)

def aux_fun : EuclideanSpace ℝ (Fin m) → Y := fun p =>
  if hp : norm p ≤ 1 then f ⟨p, by simp[hp]⟩
  else H (⟨((norm p)⁻¹ : ℝ) · p, inv_norm_smul_mem_unitClosedBall p⟩,
     norm p - 1)

def H' : (Metric.closedBall (0 : EuclideanSpace ℝ (Fin m)) 1) × ℝ
    → Y :=
  fun p => (aux_fun f H ) ((1 + p.2) · p.1)
```

I split up the `ContinuousOn` proof of `H'` in three steps. I started with a lemma, that states `aux_fun` is continuous on the ring $\{x \mid 1 \leq \|x\| \leq 2\} \subset \mathbb{R}^m$ and used this to proof continuity of `aux_fun` on the ball with radius two. The ball with radius two can be written as the union of a ball with radius one and the ring. Continuouity on both closed subsets yields continuity on their union. The continuity of `H'` follows from composing two `ContinuousOn` functions. The proofs for all desired properties of `H'` are not hard. Solving, rewirting and confirming inequalities make up big parts of them. All those results combined proof the lemma `HEP_disc_boundary`. The primed version `HEP_disc_boundary'`, is definitionally equal.

```
lemma HEP_disc_boundary : ∀ (m : ℕ),
  HEP (Metric.closedBall (0 : EuclideanSpace ℝ (Fin m)) 1) (Metric.
    sphere ⟨0, by simp⟩ 1) := by ...

lemma HEP_disc_boundary' : ∀ (m : ℕ),
  HEP' (Metric.closedBall (0 : EuclideanSpace ℝ (Fin m)) 1) (Metric.
    sphere 0 1) := HEP_disc_boundary
```

The `Metric.sphere` in Lean has type `Set` α, where α is the Type of the center (in our case the 0 in an $n-$dimenstional euclidian space). By the way `HEP` is defined, in `HEP_disc_boundary` the sphere is a subset of the closed ball and we have to add the `by simp` proof, to construct the element of the subtype. On the other hand, for `HEP'` both spaces live in the same abient space. We do not have to add a proof Lean interpretates the zero as the orign of $\mathbb{R}^m$. Here we see an advantage of the `HEP'` definition in action, since we can reduce the use of subtypes.
In the formalisation of (relative) CW-complexes, each cell is the image of a closed ball in (`Fin m`$\to \mathbb{R}$) for some `m` under a characteristic map. Same as `EuclideanSpace ℝ (Fin m)`, also `Fin m`$\to \mathbb{R}$ is a way to formalize a $m-$dimenstional real vectorspace. The relevant difference between them is, that the `EuclidianSpace` comes with the Euclidian norm, wherelse on the other version, the default norm is the maximum norm. So the closed ball, which is a preimage of a closed cell in a CW complex, is something we usually would call a cube, not

a ball. To prove the HEP for those cubes, we want to apply the fact, that a $m-$dimensional ball is homeomorphic to a cube of the same dimension. Instead of explicitly construction this homeomorphism, I wanted to obtain it from the following mathlib lemma. It states, that "if s is a convex bounded set with a nonempty interior in a real normed space, then there is a homeomorphism of the ambient space to itself that sends the interior of s to the unit open ball and the closure of s to the unit closed ball."

```
exists_homeomorph_image_interior_closure_frontier_eq_unitBall.{u_1}
    {E : Type u_1} [NormedAddCommGroup E][NormedSpace ℝ E] {s : Set
    E} (hc : Convex ℝ s) (hne : (interior s).Nonempty) (hb :
    Bornology.IsBounded s) :
  ∃ h, ↑h '' interior s = ball 0 1 ∧ ↑h '' closure s = closedBall 0
    1 ∧ ↑h '' frontier s = sphere 0 1
```

It is quite obvious, that the `Metric.closedBall`, for which we proved the HEP already, is closed, convex and nonempty and therefor satisfies all assumptions. Still we don't want to apply the above lemma directly on this. The statement would be trivial and we just obtained the identity map on the $m-$dimensional Euclidian space. Instead we first map this closed ball to `Fin m→ ℝ` via the homeomorphism

```
def homeo_euclid_max : EuclideanSpace ℝ (Fin m)  ≃ₜ (Fin m → ℝ)
    where
  toFun := fun p ↦ p
  ...
```

`homeo_euclid_max` maps each point to itself while changing the norm of the space. So the image of the `Metric.closedBall` is still the same round, convex set, even though it is not the same as a closed ball in the codomain-space. Applying the lemma to the image of the `Metric.closedBall` yields a homeomorphism form the `Fin m→ ℝ` to itsef:

```
def G : (Fin m → ℝ) ≃ₜ (Fin m → ℝ):= (
    exists_homeomorph_image_interior_closure_frontier_eq_unitBall
    convex_euclid_to_max_ball nonempty_euclid_to_max_ball
    bounded_euclid_to_max_ball).choose
```

The composition of those two homeomorphisms is especially a partial homeomorphism `EuclideanSpace ℝ (Fin m)` to `Fin m→ ℝ` with source the closed ball and target the closed cube. Now applying the lemma `PartialHomeomorph_HEP'` proves

```
lemma HEP_cube_boundary' (m : ℕ) : HEP' (closedBall (0 : (Fin m →
    ℝ)) 1) (sphere 0 1) := by ...

lemma HEP_cube_boundary (m : ℕ) : HEP (closedBall (0 : (Fin m → ℝ))
     1) (sphere ⟨0, by simp⟩ 1) := by
  exact HEP_cube_boundary' m
```

Before concluding this section, I want to point out, that here the definiton of `HEP'` matches very good the structure of the partial homeomorphis. When I first proved this result - before defining `HEP'` - for `HEP`, I had to show that the `Subtzpe.val` of a sphere in the closed ball is the same as the sphere in the ambient space and the same for the closed ball itself. With `HEP'` we avoid this subtype coercions and the proof is more straight forward. From this result the statement for `HEP` follows directely.

4.6 HEP for consecutive skeletons

Now we want to continue with the second step in the proof of the *HEP* for relative CW complexes, namely that the pair of two consecutive skeletons satisfies the *HEP*. The formalisation of this result builds on the following mathlib lemma. It replaces the theorem, which states, that the product of a quotient map with the identity on a locally compact space, is again a quotient map.

```
theorem Topology.IsQuotientMap.continuous_lift_prod_left {X₀ : Type
    u_1} {X : Type u_2} {Y : Type u_3} {Z : Type u_4} [
    TopologicalSpace X₀] [TopologicalSpace X] [TopologicalSpace Y] [
    TopologicalSpace Z] [LocallyCompactSpace Y] {f : X₀ → X} (hf :
    IsQuotientMap f) {g : X × Y → Z} (hg : Continuous fun (p : X₀ ×
     Y) => g (f p.1, p.2)) :
  Continuous g
```

We get from this theorem, that the map `g`, which is defined on the product of a quotient space with a locally compact space, is under a bunch of assumptions continuous. We want this `g` to be the retraction map $m-$skeleton $\times[0,1] \to (m-1)-$skeleton $\times[0,1]$, which yields the *HEP* for the consecutive skeletons. Let us take a closer look at which functions we need to define and which results we have to proof, in order to get there.

4.6.1 Skeleton is a quotient space

First of all we need a quotient map, called `f` in the theorem above, to the $m-$skeleton. For this we use the fact, that we can "unglue" the skeleton into the basespace and the disjoint union of closed balls (cubes to be precise) in all dimensions up to m. The identity map on the base space assembles together with the caracteristic maps on each closed ball to the map `SkeletonProjection` .

```
abbrev ungluedSkel {Y : Type*} [hY : TopologicalSpace Y] [T2Space Y]
     (X : Set Y) (C : Set Y)
    [RelCWComplex X C] (m : ℕ) :=
  C ⊕ (Σ (n : Fin m) (_ : cell X n), (closedBall (0 : Fin n → ℝ)
    1))
```

```
def SkeletonProjection {Y : Type*} [TopologicalSpace Y] [T2Space Y]
    (X : Set Y) {C : Set Y} [RelCWComplex X C] (m : ℕ) : ungluedSkel
     X C m → skeletonLT X m := Sum.elim (fun c ↦ ⟨c, by ... ⟩) (fun
     ⟨n, i, x⟩ => ⟨map n i x, by ...⟩)
```

To prove, that this indeed defines a quotient map, we have to check, that it is surjective, continuous and induces the topology on the skeleton. The skeleton is a subcomplex of the (relative) CW complex and the topology is defined by characterising the closed sets. A set of the subcomplex is closed if and only if intersected with the base space and any closed cell it is closed in those spaces.
The surjectivity proof is straight forward: A point in the skeleton lies either in the base space or an open cell. In each case there exists a preimage in the base space respectively an open ball. Also continuity is not much work, since `SkeletonProjection` is defined in terms of continuous functions on a disjoint union. Proving that the topology on the skeleton is induced, required some more effort.
We start with a set `S`, which is of Type `Set (skeletonLT X ↑m)`, and the assumption, that its preimage under the `SkeletonProjection` is closed in the unglued skeleton. The goal is to show, that the set `S` is closed in the skeleton. In mathlib the results about closed sets are always phrased for subsets of the ambient space together with the assumption, that the set is contained in the CW complex or the subcomplex. To get into this setting, we need the following lines in the proof:

```
set S' : Set Y := Subtype.val '' S with hS'def
have hS'sub : S' ⊆ (skeletonLT X m):= by
  rintro _ ⟨a, _ , rfl⟩;
  exact a.2
```

Now we can apply the lemma `Topology.RelCWComplex.isClosed_of_isClosed_inter_openCell_or_is`, which states that `S'` is closed in the relative CW complex, if the intersection with the base space is closed and for every cell either `S' ∩ openCell n j` or `S' ∩ closedCell n j` is closed. I proved the case with the base space in the lemma `hClosedBase` and the case wich cells of dimension less then m in the lemma `hClosedLT`. For higher dimensions the intersection of `S'` with the open cell is empty and hence closed.

4.6.2 construction of the retraction

The next preparation we take a look at, is the construction the map `r`. `r` is supposed to be the retraction for consecutive skeletons. In this section I want to take a closer look, at the definition of this map `r`, as well as at the related proofs related to this construction.
Note, that the `SumMap`, was called f in the mathematical section.

Same as in the mathematical proof, also in its formalisation, we don't explicitly define the retraction map `r` on the $m-$skeleton $\times[0,1]$, but instead define, what it should do, on the `ungluedSkel`$\times$ `[0,1]`. This map, will be called `SumMap` and is used in the following way, to define the `r`.

```
def r (m : ℕ) : skeletonLT X (m + 1) × unitInterval → skeletonLT X
     (m + 1) × ℝ :=
  Function.extend (fun p ↦ ((SkeletonProjection X (m + 1) p.1), p
    .2)) (sumMap m) (Prod.map id Subtype.val)
```

`Function.extend` works in a way, that whenever a point `x`$\in$ `skeletonLT X (m + 1)` has a preimage `p`$\in$ `ungluedSkel`$\times[0,1]$ under the map `(fun p`$\mapsto$ `((SkeletonProjection X (m + 1) p.1), p.2))`, then `r x = sumMap m p`. Otherwise `r x = (Prod.map id Subtype.val) x`. It is not captured in this defintion, but we know, that the map `Skeletonprojection`$\times$ `id` is surjective, so the second case of the `Function.extend` could be filled with any junk function, since it will be never applied. One could read the defintion of `r` just as, take any preimage under `SkeletonProjection` and then apply `sumMap` to it.
It might be irritating, that `r`, which is supposed to be a retraction, is formalised with the second factor of the domain only beeing the unit interval. Usually we would choose this to be the real numbers, since the retractions are defined with $\mathbb{R}$ instead of the unit interval. But observe, that the theorem `Topology.IsQuotientMap.continuous_lift_prod_left` yields `Continuous` and not `ContinuousOn`. Therefore we should not make the domain any bigger, than what is actually mathematically relevant. In the codomain we do not have this type of problems and can just choose `Z` to be the skeleton times the entire real line.

```
def cubesretract {m : ℕ} (n : Fin (m + 1)) :
    (closedBall (0 : Fin (n : ℕ) → ℝ) 1) × unitInterval →
    (closedBall (0 : Fin (n : ℕ) → ℝ) 1) × ℝ := fun p ↦
  if n = m then (r_cube n (p.1, Subtype.val p.2))
  else (p.1, Subtype.val p.2)

def sumMap (m : ℕ) :
    (ungluedSkel X C (m + 1)) × unitInterval → skeletonLT X (m+1)
    × ℝ := fun p ↦
    p.1.elim
      (fun c => (⟨c.val, memBase_memSkeletonLT X c.prop (m + 1)⟩, p
    .2))
      (fun ⟨n, i, x⟩ => (⟨map n i (cubesretract n (x,p.2)).1,
    cubesretract_mapsto_weak n i (x,p.2)⟩,
        (cubesretract n (x,p.2)).2))
```

`SumMap` is defined using `Sum.elim`, which is a "case analysis for sums that applies the appropriate function [...] after checking which constructor is present", i.e. wether the first coordinate of the applied point `p` lies in the

base space `C` or in a closed ball. In the first case, we want to do nothing, but apply the `SkeletonProjection` on this first coordinate. On the base space, this was just the "identy". So we send the a point `c : C`, to itself, but as an element of the $m-$skeleton. Since the prove, that a point from the base space lies any desired skeleton is used more often, I made this an own lemma called `memBase_memSkeletonLT`.
For the second case, that the first coordinate of the applied point `p` comes from a closed ball, we need the map `cubesretract`. `cubesretract` is the retraction map for the *HEP* of cubes relative their boundary, if the we are in the balls of top dimension and just a simple subtype coercion in all other dimension. For `cubesretract` we proved small lemmas about continuouity, where the function is fixed and in which set the image of the function lies.

```
lemma cubesretract_continuous {m : ℕ} (n : Fin (m + 1)) : Continuous
     (cubesretract n) := by ...

lemma cubesretract_fixedOn {m : ℕ} {n : Fin (m + 1)} (x : (
    closedBall (0 : Fin n → ℝ) 1) × unitInterval) :
    (n < m → cubesretract n x = (x.1, x.2.1)) ∧
    (n = m → x ∈ {z | z.2 = 0 ∨ z.1 ∈ sphere ⟨0, by simp⟩ 1 } →
    cubesretract n x = (x.1, x.2.1)) := by ...

lemma cubesretract_mapsto_lt {m : ℕ} (n : Fin (m + 1)) (i : cell X n
    ) (p : (closedBall 0 1) × ↑unitInterval) (hn : n < m) :
    (map n i) (cubesretract n p).1 ∈ skeletonLT X m ∧ (cubesretract
    n p).2 ∈ unitInterval
    := by ...

lemma cubesretract_mapsto_top {m : ℕ} (n : Fin (m + 1)) (i : cell X
    n) (p : (closedBall 0 1) × ↑unitInterval) (hn : n = m) :
    (cubesretract n p).2 = 0 ∨ (map n i) (cubesretract n p).1 ∈
    skeletonLT X m ∧ (cubesretract n p).2 ∈ unitInterval := by ...
```

So in this described second case, we first apply the map `cubesretract` to the point and then postcompose again with what `SkeletonProjection`$\times$ `id` $_{\mathbb{R}}$ would do to a such point. Since the first coordinate, also after applying `cubesretract` lies in a closed ball, `SkeletonProjection` is the same as applying the charakteristic map of that closed ball. We have to include a poof, that the image of this characteristic map actually lies in the $m-$skeleton. Also this proof in an own lemma, called `cubesretract_mapsto_weak`, which is derived from the stronger statements above.
Taking a closer look at the definition of `SumMap` it is, in the case, that the point `p` comes from the base space or a closed ball of dimension less then `m`, just the same as applying the `SkeletonProjection`$\times$ `id`$_{\mathbb{R}}$. At least when ignoring the Subtype issues. It would have been inconvenient to write this application in the definition of `SumMap`, so instead I proofed the two lemma, which show exactly this equality.

```
lemma sumMap_eq_SkelProj_C (m : ℕ) (x : C) (t : unitInterval) :
    sumMap m (Sum.inl x, t) =
    Prod.map id (Subtype.val) (SkeletonProjection X (m + 1) (Sum.inl
     x), t) := by ...

lemma sumMap_eq_SkelProj_sphereLT {m : ℕ}
    (n : Fin (m + 1)) (i : cell X n) (x' : closedBall (0 : Fin n →
    ℝ) 1) (t : unitInterval) (hx : n < m ∨ n = m ∧ x' ∈ sphere ⟨0,
    by simp⟩ 1) :
    sumMap m (Sum.inr ⟨n, ⟨i, x'⟩⟩, t) = Prod.map (SkeletonProjection
     X (m + 1)) Subtype.val (Sum.inr ⟨n, ⟨i, x'⟩⟩, t) := by ...
```

4.6.3 Continuity of the retraction

Now, that we defined all relevant maps, we can focus on how exactly we proof the continuity of `r`. As said earlier, we will get the continuity frim the theorem `Topology.IsQuotientMap.continuous_lift_prod_left`. Let's have a look at what is left to proof:

```
lemma continuousRetract (m : ℕ) : Continuous (r (X := X) m) := by
  refine (SkeletonProjectionIsQuotientMap (m +1) X).
    continuous_lift_prod_left ?_
  ...
```

After this first line of proof the current goal changes to

```
Continuous fun (p : (ungluedSkel X C m) × [0,1]) => r (
SkeletonProjection p.1, p.2)
```

Showing this boils down to two major steps. The fist one is, that `SumMap` factors through `(fun p ↦ ((SkeletonProjection X (m + 1) p.1), p.2))`. From this we can easily show that the continuity goal is equilalent to showing, that `SumMap` is continuous. Continuity of `SumMap` then is the second step. We start with

```
lemma factors (m : ℕ) : (sumMap m).FactorsThrough (fun p ↦ ((
    SkeletonProjection X (m + 1) p.1), p.2)) := by
    intro x y hxy
    ...
```

`x` and `y` are two points in `ungluedSkel× [0,1]`. Their second coordinate is the same and they have the same image under `SkeletonProjection`. Now we have to show, that `SumMap m x = SumMap m y`. With `lemma sumMap_eq_SkelProj_C` and `lemma sumMap_eq_SkelProj_sphereLT` (printed above) we already have usefull tools to prove this goal. A tricky part, that requird qute some work is:

```
lemma SkeletonProj_inj_top {Y : Type*} [TopologicalSpace Y] [T2Space
     Y] (X : Set Y) {C : Set Y} [RelCWComplex X C] {m : ℕ} {n : Fin
    (m + 1)} (hnm : ↑n = m) (i : cell X n) (x' : closedBall (0 : Fin
     n → ℝ) 1) (x_ball : x'.1 ∈ ball (0 : Fin n → ℝ) 1) (y :
    ungluedSkel X C (m + 1)) (hskel : SkeletonProjection X (m + 1) (
    Sum.inr ⟨n, ⟨i, x'⟩⟩) = SkeletonProjection X (m + 1) y) :
```

```
    y = Sum.inr ⟨n, i, x'⟩ := by
```

The lemma `SkeletonProj_inj_top` tells us, that whenever given one point `x` in an specific open ball of top dimension, and any other point `y` in `ungledSkel`. If they have the same image under `SkeletonProjection`, then `y` is the exact same point as `x`.
The prove of `factors` is long, because it is split into many different case distinctions. Begining with whether `x` lies in the base space or an closed ball. Then in each case I considered the same two options for `y` seperately. In the case, that `x` is in an closed ball, we look at the cases that it is in the top dimesnioal open cell, or not. Most of those cases themselves are quite short. In the mathematical explination it was sufficient to just look at the case, that a point is in the top dimensional cell, or not. So why do I split up the situation so much more in the formalisation? – ToDo: weil ich nicht einfach sagen kann: x in C sondern es ein c gibt,... und dann ist es einfacher, wie ich es gemacht habe
For continuity of `SumMap` we precompose the map with the homeomphism

```
def homeoProd (m : ℕ) :
  (C ⊕ (Σ (n : Fin (m + 1)) (_ : cell X n), closedBall (0 : Fin n
    → ℝ) 1)) × unitInterval ≃ₜ
  (C × unitInterval) ⊕ (Σ (n : Fin (m + 1)) (_ : cell X n),
    (closedBall (0 : Fin n → ℝ) 1) × unitInterval) := ...
```

This composite has as a domain not a product anymore, but instead a union. And on each summand the continuity goal is proven by just combining resluts proven earlier such as `cubesretract_continuous` and continuity of the characteristic maps. Since continuity is preserved under precomposition with a homeomorphism, this yields, that `SumMap` and finally also `r` is continuous.

4.6.4 Properties of the reatraction

In this section I want to conclude the proof of the *HEP* for consecutive skeletons. For the proof via the retraction criterion we have to make a small adjustment to the map `r`, which we worked with before. I mentioned in the begining, that despite the fact, the desired retraction sould have `(skeletonLT X (m + 1)) × ℝ` as a domain, `r` is just defined on the unit interval in the second component. I experienced it to be the easiest way to extend `r` by precomposing the second coordinate with the map `ProjIcc`, which was defined in the very first file. Therefore the final definition looks like below.

```
def retr_skeleton (m : ℕ) : (skeletonLT X (m + 1)) × ℝ → (
    skeletonLT X (m + 1)) × ℝ := fun p ↦
  (r m) (p.1, ⟨ProjIcc p.2, proj_mem p.2 ⟩)
```

Now it is only left to show, that `retr_skeleton` is continuous, maps to the right space and is fixed, where required. The continuity follows directely from all the work we did bevore. The claims about `mapsTo` and `fixedOn` are

first proven for `SumMap` and then transferted to `retr_skel`. All together this results in the follwoing lemma.

```
lemma HEP'_skeleton (m : ℕ) : HEP' (skeletonLT X (m + 1)).carrier (
    skeletonLT X m).carrier := by
  apply (retraction_criterion_closed (IsClosed.preimage_val (
    skeletonLT X ↑m).closed') ).2
  use retr_skeleton m
  ...
```

ToDo: man könnte hier noch auf subst eingehen, weil das ein game changer war.

4.7 HEP for finite dimensional relative CW complexes

In this last part about my formalisation we will get to the result, that finite dimensional relative CW complexes satisfy the *HEP*. In the chapter about the mathematics it was already visible, that the finite dimensional case follows straight from what we showed earlier by induction, wherelse the general case requires much more work. Due to the limited time for this thesis I had to stop at this point.
The induction looks in Lean as follows:

```
lemma HEP_skelLT_base (m : ℕ) : HEP' (skeletonLT X m) C := by
  induction m with
  | zero =>
    rw [ENat.coe_zero, skeletonLT_zero_eq_base, HEP', Subtype.
    coe_preimage_self]
    exact HEP_self
  | succ n n_ih =>
    apply HEP_trans (skeletonLT X (n + 1)).carrier  (skeletonLT X n)
     C (skeletonLT_mono
       le_self_add)  ?_ (Subcomplex.base_subset (skeletonLT X ↑n))
    n_ih
    exact HEP'_skeleton n

lemma HEP_skel_base (m : ℕ) : HEP' (skeleton X m) C := by
  rw [skeleton.congr_simp X m m rfl]
  exact HEP_skelLT_base X (m + 1)
```

The induction start, is exactly the lemma `HEP_self`, which we proved in the very begining just after defining the *HEP*. For the induction step we combine via transitivity the induction hypothesis and the *HEP* for consecutive skeletons.
During my project I used to work the whole time, as recommenden by Hannah Scholz with `SkeletonLT`, instead of `Skeleton`. And only defived the statement for `Skeleton` from this. `SkeletonLT` is especially usefull for inductions, as demonstrated above, because it also includes the case of the $(-1)-$skeleton, which would otherwise be left out.

ToDo Ausblick: In the prvious secion, where we proved the *HEP* for consecutive sekeltons, I used the lemma `retractioncriterionclosed`, where the retraction map is defined on the Type

ToDo SkelLT und skel erklären

5 Methodes and Usage of AI

6 Summary und Ausblick

References

[] *Mathlib Statistics*. URL: https://leanprover-community.github.io/mathlib_stats.html (visited on 07/02/2026).

[FRO] Lean FRO. *FRO - About us*. URL: https://lean-lang.org/fro/about/ (visited on 07/02/2026).

[Kon25] Alex Kontorovich. *The Shape of Math To Come*. 2025. arXiv: 2510.15924 [math.HO]. URL: https://arxiv.org/abs/2510.15924.

[MLS25] Alex Meiburg, Leonardo A. Lessa, and Rodolfo R. Soldati. *A Formalization of the Generalized Quantum Stein's Lemma in Lean*. 2025. arXiv: 2510.08672 [quant-ph]. URL: https://arxiv.org/abs/2510.08672.

[Sch24] Hannah Scholz. "Formalisation of CW-complexes". Bachelor's Thesis Mathematics. Rheinischen Friedrich-Wilhelms-Universität Bonn, 2024. URL: https://florisvandoorn.com/theses/HannahScholz.pdf.

[Whi18] J. H. C. Whitehead. "Combinatorial homotopy. I". In: *Bulletin (new series) of the American Mathematical Society* 55.3 (2018), pp. 213–245. ISSN: 0273-0979.